

SC4022 Network Science
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-30 — Generated for bumsclaw/ntu-cs-textbooks

Contents

1	Network Foundations & Graph Models	1
1.1	What Is a Network?	1
1.2	Degree, Paths, and Components	2
1.3	Canonical Graph Models	3
1.4	Chapter Notes	4
1.5	Exercises	4
2	Centrality Measures	6
2.1	Why Not Just Degree?	6
2.2	Distance-Based: Closeness and Betweenness	7
2.3	Eigenvector and PageRank	8
2.4	Choosing a Centrality	9
2.5	Chapter Notes	10
2.6	Exercises	10
3	Community Detection	11
3.1	What Is a Community?	11
3.2	Louvain and Label Propagation	12
3.3	Spectral Partitioning	12
3.4	Evaluating Partitions	13
3.5	Chapter Notes	14
3.6	Exercises	14
4	Random Graphs: Erdős–Rényi, Configuration & Small-World	15

4.1	Erdős–Rényi: $G(n, p)$ and $G(n, M)$	15
4.2	Configuration Model	16
4.3	Watts–Strogatz Small-World	17
4.4	Chapter Notes	18
4.5	Exercises	18
5	Scale-Free Networks & Preferential Attachment	20
5.1	Heavy Tails and Power Laws	20
5.2	Barabási–Albert Model	21
5.3	Critiquing Scale-Free Claims	22
5.4	Chapter Notes	23
5.5	Exercises	23
6	Dynamics & Spreading: SIR on Networks	25
6.1	Compartment Models on Graphs	25
6.2	Epidemic Threshold	26
6.3	Immunisation and Influence	26
6.4	Beyond SIR: Thresholds, Cascades, Opinions	28
6.5	Chapter Notes	28
6.6	Exercises	28
7	Network Robustness & Resilience	30
7.1	Measuring Robustness: $S(f)$	30
7.2	Percolation Thresholds	31
7.3	Robust Yet Fragile and Hardening	31
7.4	Chapter Notes	33
7.5	Exercises	33
8	Applications: Social, Biological & Technological Networks	34
8.1	Social Networks	34
8.2	Biological Networks	35
8.3	Technological Networks	35

8.4 Universal vs Specific and Capstone 36

8.5 Chapter Notes 37

8.6 Exercises 37

Chapter 1

Network Foundations & Graph Models

“A network is more than its parts.” — From a handful of friends to the entire Web, the pattern of connections shapes what the whole system can do. Before measuring importance or modelling growth, we need a language for that pattern.

From zero: we build here, no prior graph theory needed. We assume only sets and basic counting. Every notion—node, edge, degree, path, component—is motivated by a picture, then defined, then exercised. No prior bachelor’s knowledge is required.

Learning Outcomes

After this chapter you will be able to:

- (L1) define graphs, directed, weighted and bipartite variants and their adjacency representations;
- (L2) compute degree, degree distribution, and handshaking properties on small examples;
- (L3) explain paths, components, diameter, and clustering coefficient intuitively and formally;
- (L4) classify canonical models: lattice, tree, complete, star, bipartite, and their trade-offs;
- (L5) build and manipulate graphs in Python (NetworkX) choosing list vs matrix.

1.1 What Is a Network?

Intuition. Picture four people: Alice knows Bob and Carol; Bob knows Carol and Dave. Draw a dot for each person, a line for each acquaintance. The dots-and-lines picture *is* the network. The dots are “where things are”; the lines are “what connects to what.” That is all a network is—yet from this spare picture we can ask rich questions: who is most connected? how far does news travel? does the network split into islands?

Formal view. A graph $G = (V, E)$ is a finite set V of *nodes* (vertices) and a set E of *edges* linking them. For undirected graphs an edge is an unordered pair $\{u, v\}$ with $u \neq v$ (no self-loops for now). We write $n = |V|$, $m = |E|$. Variants add direction or weight.

Definition 1.1 (Graph and variants). An *undirected simple graph* is $G = (V, E)$ with $E \subseteq \binom{V}{2}$. A *directed graph* has $E \subseteq V \times V$ (ordered pairs, arrows). A *weighted graph* adds $w : E \rightarrow \mathbb{R}$ (e.g. distance, strength). A *bipartite graph* has $V = V_1 \cup V_2$, $V_1 \cap V_2 = \emptyset$, and every edge joins V_1 to V_2 . A *multigraph* allows multiple edges; a *self-loop* is $\{v, v\}$.

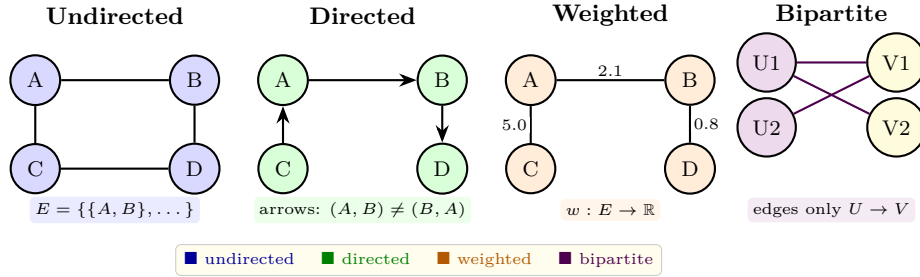


Figure 1.1: Four graph flavours. Colour distinguishes edge meaning. Undirected edges are symmetric friendships; directed edges are follows or citations; weights encode distance or strength; bipartite edges cross the partition.

Two representations compete. The *adjacency matrix* A is $n \times n$ with $A_{ij} = 1$ if $\{i, j\} \in E$ (or weight). The *adjacency list* stores for each node its neighbours. Matrix checks “is there an edge?” in $O(1)$ but uses $\Theta(n^2)$ memory; list uses $\Theta(n + m)$ and iterates neighbours fast—best for sparse real networks where $m \ll n^2$.

1.2 Degree, Paths, and Components

Intuition. If you count the lines touching a dot, you know how sociable that person is. If you can hop dot-to-dot from s to t , they are in the same island; if not, the network is split. The shortest hop-count is a distance.

Definition 1.2 (Degree, path, component). The *degree* $k_v = |N(v)|$ counts neighbours of v . The *degree distribution* $P(k)$ is the fraction of nodes with degree k ; the *average degree* is $\langle k \rangle = 2m/n$. A *path* is a sequence $v_0, v_1, \dots, v_\ell$ with $\{v_i, v_{i+1}\} \in E$. Its *length* is ℓ (edges). The *distance* $d(s, t)$ is the shortest path length (or ∞ if disconnected). A *connected component* is a maximal connected subgraph. The *diameter* is $\max_{s, t} d(s, t)$ over the largest component.

The *handshaking lemma* is a first theorem: $\sum_v k_v = 2m$ because each edge contributes one to the degree at each end. Average degree follows immediately. On Fig. 1.2, node 3 has degree 3 (most connected), nodes 1 and 8 have degree 1.

Clustering: are my friends friends? The *clustering coefficient* formalises triangle density. Locally, $C_v = (\text{edges among neighbours of } v) / \binom{k_v}{2}$ (or 0 if $k_v < 2$). Globally, $C = (1/n) \sum_v C_v$ or transitivity $3 \times \text{triangles} / \text{connected triples}$. A network where “friends of friends are friends” has high C .

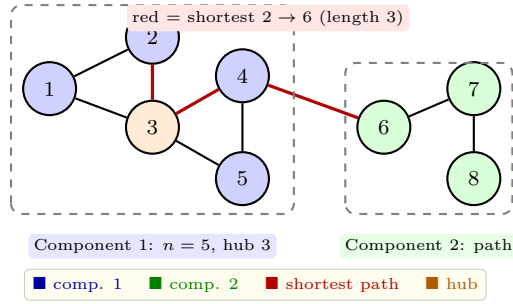


Figure 1.2: Paths and components on an 8-node graph. Two components: $\{1, 2, 3, 4, 5\}$ and $\{6, 7, 8\}$. The shortest path $2 \rightarrow 6$ has length 3 (red). Node 3 is the degree hub. Diameter of the left component is 3.

Example 1.1 (Small worked clustering). In Fig. 1.2, neighbours of node 3 are $\{1, 2, 5\}$. Among them, edges present: $\{1, 2\}$ only, so $C_3 = 1/\binom{3}{2} = 1/3$. Node 1 has neighbours $\{2, 3\}$ with edge $\{2, 3\}$ present, so $C_1 = 1$. Node 6 has neighbours $\{4, 7\}$ with no edge between them, so $C_6 = 0$. Average $C \approx 0.27$ for this graph.

1.3 Canonical Graph Models

Intuition. Before asking “what does Facebook look like?” we describe extremes to calibrate intuition: a ring where everyone knows neighbours (ordered), a tree with no cycles (hierarchical), a clique where everyone knows everyone (dense), and a star centred on one hub (unequal). Real networks sit between these.

Definition 1.3 (Canonical models). A *path* P_n and *cycle* C_n have n nodes in a line or ring. A *complete graph* K_n has all $\binom{n}{2}$ edges ($k_v = n - 1$). A *star* S_n has one centre linked to $n - 1$ leaves. A *tree* has $n - 1$ edges, no cycles. A *regular lattice* links each node to its k nearest neighbours on a ring. A *bipartite complete* $K_{a,b}$ links every a on the left to every b on the right.

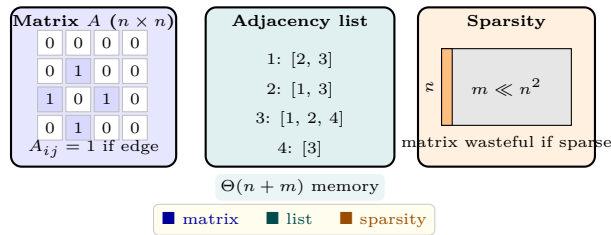


Figure 1.3: Matrix vs list representation. Matrix is dense $n \times n$ (left); list stores only neighbours (centre). For sparse networks ($m \ll n^2$) the list wins by far (right).

Listing 1.1: Building and inspecting graphs with NetworkX: representations and basic measures.

```

1 import networkx as nx
2
3 G = nx.karate_club_graph()           # Zachary karate: 34 nodes, 78 edges
4 print(f"n={G.number_of_nodes()} m={G.number_of_edges()}")
5     f" <k>={2*G.number_of_edges()/G.number_of_nodes():.2f}"
6
7 # Degree and distribution
8 degrees = [d for _, d in G.degree()]
9 print("max degree:", max(degrees), "min degree:", min(degrees))
10
11 # Clustering and diameter (largest component)

```

```

12 print("avg clustering:", round(nx.average_clustering(G), 3))
13 print("diameter:", nx.diameter(G))
14
15 # Adjacency matrix vs list (sparse check)
16 import numpy as np
17 A = nx.to_numpy_array(G)          # n x n dense
18 print("matrix shape:", A.shape, "sparsity:", (A==0).sum()/A.size)
19 print("list for node 0:", list(G.neighbors(0))[:5])

```

Listing 1.2: Canonical models: lattice, star, and bipartite gallery.

```

1 import matplotlib.pyplot as plt
2
3 models = {
4     "Cycle C8": nx.cycle_graph(8),
5     "Star S7": nx.star_graph(6),
6     "Complete K5": nx.complete_graph(5),
7     "Tree": nx.balanced_tree(r=2, h=3),
8 }
9 for name, H in models.items():
10     print(name, f"n={H.number_of_nodes()} m={H.number_of_edges()}")
11         f" diam={nx.diameter(H) if nx.is_connected(H) else 'inf'}"
12         f" C={nx.average_clustering(H):.2f}")
13 # Bipartite: 3 producers - 4 consumers
14 B = nx.complete_bipartite_graph(3, 4)
15 print("K3,4 edges:", B.number_of_edges(), "is_bipartite:", nx.is_bipartite(B))

```

1.4 Chapter Notes

Real-network thinking begins with Euler (1736) on the Königsberg bridges—nodes and edges abstracting geography. Modern Network Science (Barabási, 2016; Newman, 2018) systematised degree distributions, clustering, and diameter as the three lenses used in this chapter. For sparse-network algorithms, adjacency lists with BFS/DFS remain the workhorse from SC2001; matrix methods dominate spectral analyses (Ch. 3). The Zachary karate club dataset used above will recur throughout this book as a running example.

1.5 Exercises

- E1.1 **Handshake lemma.** A graph has degree sequence $[3, 2, 2, 1, 1, 1]$. How many edges? Draw one such graph or prove none exists. State and prove $\sum_v k_v = 2m$ and deduce $n\langle k \rangle = 2m$.
- E1.2 **Matrix vs list.** For $n = 10^5$, $m = 5 \times 10^5$, estimate memory for adjacency matrix (1 byte per entry, symmetric) vs adjacency list (4 bytes per int + overhead). Which fits in 1 GB? At what m does matrix become competitive?
- E1.3 **Clustering by hand.** On a 5-node house graph (square 1-2-3-4-1 plus roof 2-5-3), compute C_v for each v and the average C . Compute transitivity and compare. Which node has highest local clustering?

- E1.4 **Diameter chase.** Find the diameter of: (a) P_n , (b) C_n , (c) K_n , (d) star S_n , (e) $K_{2,5}$ (use BFS reasoning, no code). Which grows with n and which stays constant?
- E1.5 **Build and verify.** Using NetworkX, build a 12-node ring lattice where each node links to its 2 nearest neighbours on each side ($k = 4$). Report m , average clustering, and diameter. Then rewired 2 random edges and report how each metric changes.

Chapter 2

Centrality Measures

“Who matters most?” — In a friendship network, popularity differs from being a bridge; in the Web, a page linked by important pages is important. Different notions of “central” answer different questions.

From zero: no prior centrality needed. We assume only Ch. 1 (graph, degree, path, distance). Each measure is motivated by a story—counting friends, being near everyone, lying on many paths, being endorsed by the important—then defined, computed, and compared.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain why degree alone misranks nodes when position matters;
- (L2) compute closeness and betweenness centralities on small graphs by hand;
- (L3) define eigenvector centrality and derive it from the adjacency spectrum;
- (L4) formulate PageRank as a random walk with teleportation and compute its stationary distribution;
- (L5) compare centrality rankings (correlation, rank disagreement) and select the right measure for a task.

2.1 Why Not Just Degree?

Intuition. In a star, the centre has degree 5 and every leaf has degree 1—degree ranks them correctly. But attach two stars by a single bridge node (Fig. 2.1). The bridge has degree 2, far below each star centre (5), yet removing the bridge disconnects the network while removing a leaf does nothing. “Important” depends on *where* you sit, not just how many neighbours you have.

Consider also “being close to everyone”: a node in the middle of a chain reaches all others in few hops, while a leaf at the end needs many hops, even if both have degree 2 and 1 respectively. And “being endorsed by the

important”: a junior researcher cited by two Nobel laureates may outrank one cited by ten undergraduates. These stories give closeness, betweenness, and eigenvector ideas.

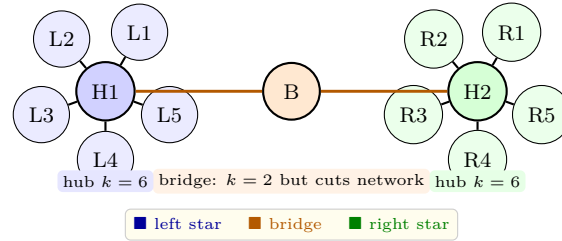


Figure 2.1: Degree misranks when bridging matters. Both hubs have degree 6, the bridge has degree 2, yet the bridge lies on every path between left and right halves. Betweenness captures this; degree does not.

Definition 2.1 (Degree centrality). For node v , $C_D(v) = k_v$ (or normalised $k_v/(n-1)$). It is local—cost $O(m)$ for all nodes—and a useful baseline, but blind to global position.

2.2 Distance-Based: Closeness and Betweenness

Intuition for closeness. If you must send a message to everyone, you want small total (or average) distance. A centre of a chain is closer on average than an endpoint.

Definition 2.2 (Closeness and harmonic). Let $d(v, u)$ be shortest-path distance. Then

$$C_C(v) = \frac{n-1}{\sum_{u \neq v} d(v, u)} \quad (\text{normalised closeness}). \quad (2.1)$$

If the graph is disconnected we use *harmonic centrality* $C_H(v) = \sum_{u \neq v} 1/d(v, u)$ with $1/\infty := 0$, which handles ∞ gracefully.

Intuition for betweenness. Stand on a node and count how many shortest paths you intercept. A bridge between communities intercepts many; a leaf intercepts none.

Definition 2.3 (Betweenness). Let σ_{st} be the number of shortest $s \rightarrow t$ paths and $\sigma_{st}(v)$ those passing through v ($s \neq v \neq t$). Then

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}. \quad (2.2)$$

Normalise by $\binom{n-1}{2}$ for undirected graphs to get $[0, 1]$. Brandes’ algorithm computes all C_B in $O(nm)$ (unweighted) via BFS from each source.

Example 2.1 (Hand computation on a 5-node chain). Chain 1–2–3–4–5. Distances from node 3: 1, 1, 2, 2 so $\sum d = 6$ and $C_C(3) = 4/6 = 0.667$. Endpoint 1 has distances 1, 2, 3, 4, sum 10, $C_C(1) = 0.4$ —less central. Betweenness: node 3 lies on paths $1 \rightarrow 4, 1 \rightarrow 5, 2 \rightarrow 4, 2 \rightarrow 5$ (4 pairs) plus symmetric, giving $C_B(3) = 6$ unnormalised (highest); endpoints have 0.

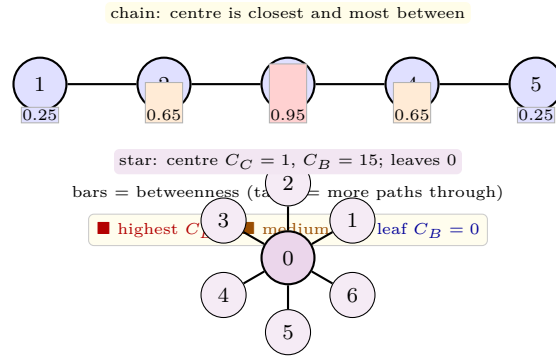


Figure 2.2: Closeness and betweenness on a chain and a star. On the chain, the middle node maximises both; on the star, the centre dominates utterly. Bars sketch unnormalised C_B .

2.3 Eigenvector and PageRank

Intuition for eigenvector. “Important if linked by important nodes.” This is self-referential: let x_v be importance of v ; then $x_v \propto \sum_{u \in N(v)} x_u$. In matrix form $Ax = \lambda x$ — x is an eigenvector of A .

Definition 2.4 (Eigenvector centrality). Let A be the adjacency matrix. The *eigenvector centrality* is the entrywise non-negative eigenvector for the largest eigenvalue λ_1 :

$$x_v = \frac{1}{\lambda_1} \sum_u A_{vu} x_u, \quad \|x\|_2 = 1, \quad x_v \geq 0. \quad (2.3)$$

Perron–Frobenius guarantees existence for connected (irreducible) graphs. Power iteration $x^{(t+1)} \propto Ax^{(t)}$ converges to it.

Intuition for PageRank. Eigenvector fails on directed graphs with sinks (a node with out-degree 0 traps importance). PageRank imagines a random surfer: with probability α follow a random out-link, with probability $1 - \alpha$ teleport to a random node. This makes the walk irreducible and aperiodic, so a unique stationary distribution exists—that distribution *is* PageRank.

Definition 2.5 (PageRank). For directed graph with out-degree k_u^{out} , transition $P_{uv} = A_{uv}/k_u^{\text{out}}$ (or $1/n$ if $k_u^{\text{out}} = 0$). With damping $\alpha \in (0, 1)$ (usually 0.85) and teleportation matrix $J_{uv} = 1/n$,

$$G = \alpha P + (1 - \alpha)J, \quad \pi = \pi G, \quad \sum_v \pi_v = 1, \quad \pi_v \geq 0. \quad (2.4)$$

π is the PageRank vector; π_v is the long-run fraction of time at v .

Listing 2.1: Centrality zoo on Zachary karate club: degree, closeness, betweenness, eigenvector, PageRank.

```

1 import networkx as nx
2
3 G = nx.karate_club_graph()
4 # All centralities (normalised where applicable)
5 deg = nx.degree_centrality(G)
6 clo = nx.closeness_centrality(G)
7 bet = nx.betweenness_centrality(G, normalized=True)
8 eig = nx.eigenvector_centrality(G, max_iter=500)

```

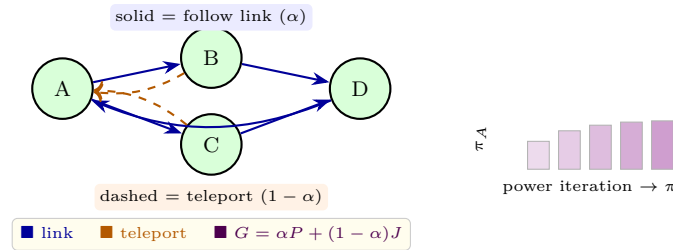


Figure 2.3: PageRank as a random walk with teleportation. Solid arrows are link-following; dashed are random jumps that guarantee ergodicity. Right: power iteration converges to the stationary distribution π .

```

9 pr = nx.pagerank(G, alpha=0.85)
10
11 def top(d, k=3): return sorted(d, key=d.get, reverse=True)[:k]
12 print("degree:      ", top(deg), [round(deg[v],2) for v in top(deg)])
13 print("closeness:   ", top(clo), [round(clo[v],2) for v in top(clo)])
14 print("betweenness:", top(bet), [round(bet[v],3) for v in top(bet)])
15 print("eigenvector:", top(eig), [round(eig[v],3) for v in top(eig)])
16 print("pagerank:    ", top(pr), [round(pr[v],3) for v in top(pr)])
17 # Spearman correlation: do rankings agree?
18 import scipy.stats as st
19 print("deg vs bet rho:", round(st.spearmanr(list(deg.values()),
20     list(bet.values()))).correlation, 2))

```

Listing 2.2: Betweenness reveals bridges: star-of-stars where degree misleads.

```

1 import networkx as nx
2 # Two cliques joined by a bridge node
3 G = nx.Graph()
4 G.add_edges_from([(0,1),(0,2),(1,2)]) # left triangle
5 G.add_edges_from([(3,4),(3,5),(4,5)]) # right triangle
6 G.add_edge(2, 3) # bridge edge 2-3
7
8 print("degrees:", dict(G.degree()))
9 print("betweenness:", {v: round(b,3) for v,b in
10     nx.betweenness centrality(G).items()})
11 print("closeness:", {v: round(c,3) for v,c in
12     nx.closeness centrality(G).items()})
13 # Node 2 and 3 have modest degree but highest betweenness

```

2.4 Choosing a Centrality

No single measure is “correct.” Degree is cheap and local—good for immediate risk (hub vaccination). Closeness suits broadcast reach (place a warehouse). Betweenness suits bottlenecks (protect a bridge router). Eigenvector/PageRank suits endorsement networks (Web search, citation). A practical workflow: compute several, check rank correlation (Spearman ρ), inspect disagreements, and validate against an external outcome (e.g. does high PageRank predict traffic?).

2.5 Chapter Notes

Centrality thinking began with Bavelas (1948) and Freeman (1977) formalising betweenness and closeness for social networks. Eigenvector centrality was refined by Bonacich (1972); PageRank (Brin & Page, 1998) made it the backbone of Web search. Brandes (2001) gave the $O(nm)$ betweenness algorithm still in use. Modern caution: centralities are highly correlated on many real graphs—reporting all five without noting redundancy is a common pitfall.

2.6 Exercises

- E2.1 **Degree vs position.** Draw the “lollipop”: a K_4 (nodes 1–4) plus a chain 4–5–6. Rank all nodes by degree, then by betweenness (reason, no code). Which node is second-highest in betweenness despite modest degree? Explain.
- E2.2 **Closeness by hand.** Compute $C_C(v)$ for every v in the 4-node star (centre 1 + leaves 2, 3, 4). Then compute harmonic centrality and verify that ordering agrees but values differ.
- E2.3 **Eigenvector iteration.** For the triangle graph K_3 , start $x^{(0)} = [1, 0, 0]$ and do two steps of $x^{(t+1)} \propto Ax^{(t)}$ (normalise to sum 1). What limit do you expect and why? What happens if the graph has two components?
- E2.4 **PageRank damping.** On a directed 3-cycle $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$, what is PageRank for any α ? Add a dangling node 4 with edge $3 \rightarrow 4$ (no out-links from 4): how does π_4 vary with α ? Explain teleportation’s role.
- E2.5 **Correlated rankings.** Using NetworkX on the karate club, compute Spearman ρ for every pair among degree, betweenness, and PageRank. Report the most and least correlated pair and inspect which nodes cause rank disagreements.

Chapter 3

Community Detection

“Birds of a feather flock together.” — Social circles, protein complexes, and Web topics all appear as clusters: many links inside, few between. Finding those clusters automatically is community detection.

From zero: we build here, no clustering background needed. We assume only Ch. 1–2 (graph, degree, paths). Every notion—community, modularity, spectral cut—is introduced by picture, then defined, then exercised.

Learning Outcomes

After this chapter you will be able to:

- (L1) define community structure and explain why “dense inside, sparse outside” matters;
- (L2) formulate modularity Q and compute it on small partitions by hand;
- (L3) run one step of Louvain greedy optimisation and trace label propagation;
- (L4) apply spectral bisection via the graph Laplacian and Fiedler vector;
- (L5) evaluate partitions with NMI, adjusted Rand, and conductance.

3.1 What Is a Community?

Intuition. Look at Fig. 3.1: two triangles linked by a single edge. Inside each triangle every pair is connected (dense); between triangles there is one bridge (sparse). If you delete that bridge, the network splits cleanly. A *community* is such a piece—more edges than you would expect by chance. The challenge: formalise “than expected” so a computer can search for it.

We need a *null model*: what would the edge pattern look like if there were no communities but degrees were preserved? The *configuration model* (Ch. 4) randomises edges while keeping each node’s degree, so an edge

between high-degree nodes is less surprising than one between low-degree nodes. Modularity counts surprise.

Definition 3.1 (Modularity Q). For partition $\mathcal{C} = \{c_i\}$ where c_i is the community of node i , with adjacency A_{ij} and $m = |E|$,

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j), \quad (3.1)$$

where $\delta(c_i, c_j) = 1$ if $c_i = c_j$, else 0. Equivalently $Q = \sum_c (e_{cc} - a_c^2)$ with e_{cc} = fraction of edges inside c and a_c = fraction of edge-ends in c . Range $Q \in [-1/2, 1]$; $Q > 0.3$ often indicates meaningful communities; $Q \approx 0$ suggests none.

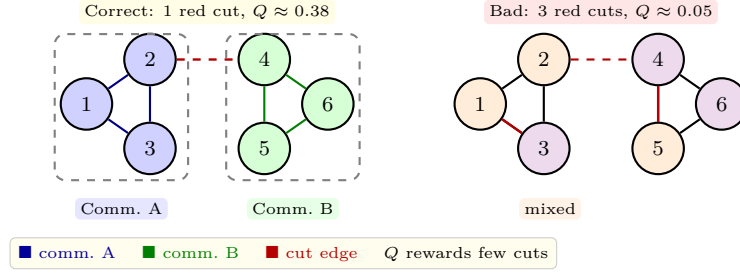


Figure 3.1: Two partitions of the same 6-node graph. Left: each triangle is a community—one cut edge, high modularity. Right: mixing nodes across triangles—three cuts, low Q . Modularity formalises “surprisingly dense inside.”

Example 3.1 (Modularity by hand). Take left partition in Fig. 3.1: $m = 7$ edges, $k_i = 2$ for leaves and 3 for the two bridge nodes (2 and 4). For community $A = \{1, 2, 3\}$, $e_{AA} = 3/7$, $a_A = (2 + 3 + 2)/(2 \cdot 7) = 7/14 = 0.5$; similarly $a_B = 0.5$, $e_{BB} = 3/7$. So $Q = (3/7 - 0.25) + (3/7 - 0.25) \approx 0.36$. Right partition splits triangles, giving $Q \approx 0.05$ —much worse, as expected.

3.2 Louvain and Label Propagation

Intuition for Louvain. Start with each node alone. Repeatedly try moving a node to a neighbour’s community if Q increases; when no move helps, collapse each community into a super-node and repeat. This greedy hill-climbing is fast ($O(m)$ per pass) and finds high- Q partitions on large graphs. The Leiden variant adds a refinement step to guarantee connected communities.

Intuition for label propagation. Give each node a unique label, then repeatedly let each node adopt the most frequent label among neighbours (ties broken randomly). Dense regions quickly agree on one label; boundaries oscillate. It is near-linear and needs no parameter, but results vary with update order.

3.3 Spectral Partitioning

Intuition. Place nodes on a line so connected nodes stay close; then cut the line in the middle. The Laplacian $L = D - A$ (degree matrix minus adjacency) measures “stretch”; its second-smallest eigenvector (Fiedler vector) gives the best 1-D placement.

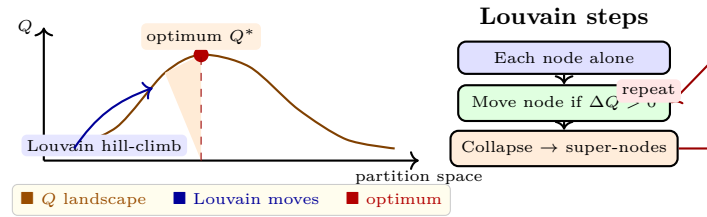


Figure 3.2: Modularity optimisation as hill-climbing. Left: Q rises to an optimum then falls as partitions over-merge. Louvain moves single nodes greedily, collapses communities, and repeats. Leiden adds refinement to avoid disconnected communities.

Definition 3.2 (Graph Laplacian and Fiedler cut). Let $D = \text{diag}(k_1, \dots, k_n)$. The *Laplacian* is $L = D - A$, symmetric positive semidefinite, with eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$. The *Fiedler vector* f satisfies $Lf = \lambda_2 f$, $\sum_i f_i = 0$. Spectral bisection puts i in S if $f_i \geq 0$, else $\bar{S}$. Small λ_2 signals a good cut (nearly disconnected); large λ_2 means well-connected (expander).

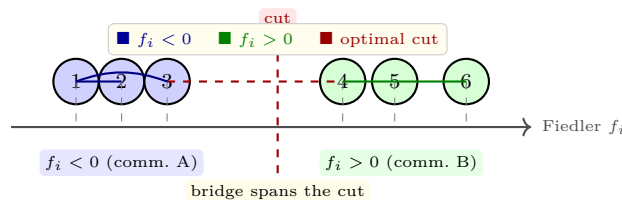


Figure 3.3: Spectral embedding: the Fiedler vector places nodes on a line; sign gives the bisection. Nodes 1–3 fall left of zero, 4–6 right; the bridge edge 3–4 is the only one crossing, so conductance is minimal.

3.4 Evaluating Partitions

When ground-truth communities are known (e.g. planted partition or karate labels), compare with *normalised mutual information* (NMI, 1 = identical) or *adjusted Rand index* (ARI). Without ground truth, use *conductance* $\phi(S) = |\text{cut}(S, \bar{S})| / \min(\text{vol}(S), \text{vol}(\bar{S}))$ (small = good community) and coverage. Modularity itself is a score but should not be the sole evaluation—it has a resolution limit: it merges small communities in large graphs.

Listing 3.1: Louvain communities and modularity on the karate club.

```

1 import networkx as nx
2 import community as community_louvain # python-louvain
3
4 G = nx.karate_club_graph()
5 part = community_louvain.best_partition(G) # dict node -> comm id
6 mod = community_louvain.modularity(part, G)
7 print(f"communities: {len(set(part.values()))} Q={mod:.3f}")
8 # Show sizes
9 from collections import Counter
10 print(Counter(part.values()))
11 # Louvain step intuition: try moving node 0 and check delta Q

```

Listing 3.2: Spectral bisection via the Laplacian Fiedler vector.

```

1 import networkx as nx
2 import numpy as np
3

```

```

4 | G = nx.karate_club_graph()
5 | L = nx.laplacian_matrix(G).toarray()
6 | w, V = np.linalg.eigh(L)           # eigenvalues sorted
7 | fiedler = V[:, 1]                 # second smallest
8 | # Bisection by sign
9 | comm = {i: int(fiedler[i] >= 0) for i in range(len(fiedler))}
10 | # Conductance of this cut
11 | S = {i for i, c in comm.items() if c==0}
12 | cut = sum(1 for u, v in G.edges() if (u in S) != (v in S))
13 | vol_S = sum(dict(G.degree(S)).values())
14 | vol_rest = 2*G.number_of_edges() - vol_S
15 | print(f"lambda2={w[1]:.3f} cut={cut} conductance={cut/min(vol_S, vol_rest):.3f}")

```

Remark 3.1 (Overlapping communities). A node may belong to several groups (family, lab, sports). Extensions like clique percolation or mixed-membership SBM allow c_i to be a distribution over communities. Louvain and spectral cuts are disjoint; overlapping scores need generalised modularity or NMI variants.

3.5 Chapter Notes

Modularity was introduced by Newman & Girvan (2004) and Newman (2006); Louvain by Blondel et al. (2008) and Leiden by Traag et al. (2019). Spectral partitioning originates with Fiedler (1973) and the Cheeger inequality linking λ_2 to conductance. The resolution limit (Fortunato & Barthélemy, 2007) warns that Q optimisation merges communities smaller than $\sqrt{2m}$. Overlapping methods (clique percolation, MMSB) address nodes belonging to many groups.

3.6 Exercises

- E3.1 **Modularity by hand.** For Fig. 3.1 left (two triangles, one bridge, $m = 7$), compute Q for: (a) the correct two-community split, (b) all nodes in one community, (c) each node alone. Which is highest? Why does (b) give $Q = 0$?
- E3.2 **Louvain trace.** Start with 6 singleton communities on Fig. 3.1. Consider moving node 2 into community $\{1, 3\}$: compute ΔQ (formula or brute force with code). Would Louvain accept this move on the first pass?
- E3.3 **Fiedler sign.** Build the 6-node two-triangle graph in NetworkX, compute Laplacian eigenvalues, and verify $\lambda_2 > 0$ (connected). Report the Fiedler vector and check that its sign matches the natural bisection.
- E3.4 **NMI vs modularity.** On the karate club, compare Louvain’s partition to the ground-truth “Mr. Hi vs John A” split (node attribute ‘club’). Compute NMI (sklearn) and modularity for both. Which score favours which partition and why?
- E3.5 **Resolution limit.** Consider a ring of k cliques each of size c , linked in a cycle by one edge between successive cliques. For large k , Newman shows Louvain merges adjacent cliques despite clear structure. Explain why Q prefers merging when $c < \sqrt{2m}$ and test with $c = 4$, $k = 10$ in code.

Chapter 4

Random Graphs: Erdős–Rényi, Configuration & Small-World

“What does a typical network look like?” — If you toss a coin for every possible edge, what graph appears? When does it become connected? How can we build a random graph with a prescribed degree sequence, or one that is both clustered and short?

From zero: we build here, no random-graph background needed. We assume only Ch. 1 (graph, degree, clustering, path) and basic probability (SC2000: Bernoulli, binomial, expectation). Every model is motivated by a picture, then defined, then exercised.

Learning Outcomes

After this chapter you will be able to:

- (L1) define $G(n, p)$ and $G(n, M)$, derive degree distribution and $\langle k \rangle$;
- (L2) explain the giant-component phase transition at $p = 1/n$ (or $\langle k \rangle = 1$);
- (L3) construct the configuration model and generate graphs with arbitrary $P(k)$;
- (L4) generate Watts–Strogatz small-world graphs and quantify the clustering vs path trade-off;
- (L5) test whether a real network’s feature is significant against a null (random) model.

4.1 Erdős–Rényi: $G(n, p)$ and $G(n, M)$

Intuition. Flip a biased coin with heads probability p for each of the $\binom{n}{2}$ possible undirected edges, independently. Heads = keep the edge. The resulting graph is random, yet its typical properties are sharply predictable: for tiny p it is dust (isolated nodes); as p grows, a giant connected piece suddenly appears, then the whole graph connects.

Definition 4.1 (Erdős–Rényi models). $G(n, p)$: n labelled nodes, each edge present independently with probability p . $G(n, M)$: uniform over all graphs with exactly M edges ($0 \leq M \leq \binom{n}{2}$). They are asymptotically equivalent when $M = p\binom{n}{2}$ and n large.

Degree distribution. Fix a node v ; its degree $k_v \sim \text{Binomial}(n-1, p)$ because each of the $n-1$ potential neighbours is present with prob. p :

$$\Pr(k_v = k) = \binom{n-1}{k} p^k (1-p)^{n-1-k}, \quad \langle k \rangle = (n-1)p \approx np. \quad (4.1)$$

For sparse graphs with $\langle k \rangle$ fixed and $n \rightarrow \infty$, $p = \langle k \rangle/n$ and the binomial tends to $\text{Poisson}(\langle k \rangle)$:

$$P(k) \approx e^{-\langle k \rangle} \langle k \rangle^k / k!. \quad (4.2)$$

So ER degrees are concentrated near the mean, unlike the heavy tails of Ch. 5.

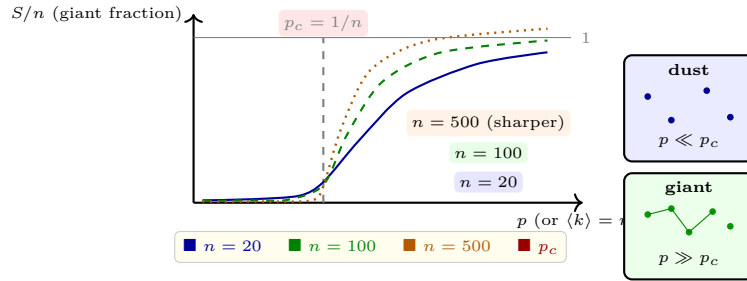


Figure 4.1: Giant-component emergence in $G(n, p)$. For $p < 1/n$ the largest component is $O(\log n)$ (dust); for $p > 1/n$ a component of size $\Theta(n)$ appears. The transition sharpens as n grows—a phase transition.

Theorem 4.1 (Giant component threshold). In $G(n, p)$ with $p = \langle k \rangle/n$, if $\langle k \rangle < 1$ then with high probability all components have size $O(\log n)$; if $\langle k \rangle > 1$ there is a unique giant component of size Sn with $S = 1 - e^{-\langle k \rangle S}$, and the rest are $O(\log n)$. The critical point is $\langle k \rangle = 1$ ($p = 1/n$). Moreover $G(n, p)$ becomes connected when $p \approx \ln n/n$.

Idea. Branching-process sketch: explore from a node, each neighbour spawns $\approx \text{Poisson}(\langle k \rangle)$ new neighbours. If mean offspring < 1 , the process dies quickly (subcritical); if > 1 , it survives with positive probability, giving the giant. Solving $S = 1 - e^{-\langle k \rangle S}$ yields the survival fraction. Connectivity needs the last isolated node to attach, which occurs at $\ln n/n$. $\square$ $\square$

4.2 Configuration Model

Intuition. ER fixes only n and p , so degrees are Poisson. Real networks have arbitrary $P(k)$ (often heavy-tailed). How to generate a random graph with a *given* degree sequence $(k_1, \dots, k_n)$? Give each node k_i half-edges (stubs), then pair stubs uniformly at random.

Definition 4.2 (Configuration model). Given degree sequence $(k_1, \dots, k_n)$ with $\sum_i k_i = 2m$ even: create k_i stubs at node i , then repeatedly pick two unmatched stubs uniformly and connect them. Optionally erase self-loops and multi-edges (if $k_{\max}^2 \ll 2m$ they are rare).

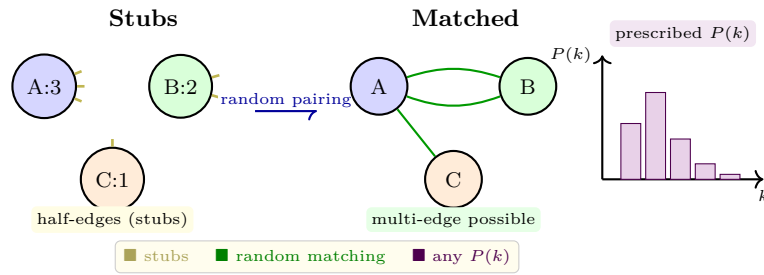


Figure 4.2: Configuration model: create k_i stubs per node (left), pair uniformly (centre) to realise any prescribed degree distribution (right). For sparse graphs self-loops and multi-edges are $O(1/n)$.

A key null-model use: compute modularity or clustering on the real graph, then compare to the configuration ensemble with the same degrees. If the real Q far exceeds the random Q , communities are significant.

4.3 Watts–Strogatz Small-World

Intuition. ER has short paths ($\sim \log n$) but negligible clustering ($C \sim \langle k \rangle / n$). A ring lattice has high clustering ($C \approx 3/4$ for $k = 4$) but long paths ($\sim n/2k$). Real networks have *both*: e.g. social nets are clustered yet six degrees apart. Watts and Strogatz rewired a lattice slightly to get both.

Definition 4.3 (Watts–Strogatz model). Start with a ring of n nodes, each linked to $k/2$ neighbours on each side (k even). For each edge (i, j) , with probability p rewire it to (i, ℓ) where ℓ is random (avoid duplicates/self-loops). $p = 0$ is the lattice; $p = 1$ is random-like; intermediate $p \sim 0.01$ – 0.1 gives high C and low L —the small-world regime.

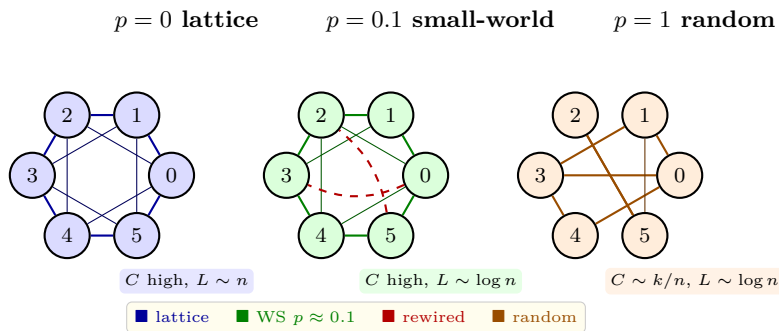


Figure 4.3: Watts–Strogatz rewiring. A lattice ($p = 0$) has high clustering but long paths. Rewiring a few edges (red dashed, $p \approx 0.1$) drops L dramatically while C stays high—the small-world. Full rewiring ($p = 1$) randomises.

Listing 4.1: ER giant-component hunt: sweep p and track largest component.

```

1 import networkx as nx
2 import numpy as np
3
4 def giant_fraction(n, p, trials=20):
5     fracs = []
6     for _ in range(trials):
7         G = nx.erdos_renyi_graph(n, p)
8         largest = max(len(c) for c in nx.connected_components(G)) if G else 0
9         fracs.append(largest / n)

```

```

10     return np.mean(fracs)
11
12 for p in [0.005, 0.01, 0.02, 0.05]:
13     print(f"p={p:.3f} <k>={p*99:.1f} S~{giant_fraction(100,p):.2f}")
14 # Watch S jump near p=1/n=0.01

```

Listing 4.2: Configuration and Watts–Strogatz: prescribed degrees and the small-world trade-off.

```

1 import networkx as nx
2
3 # Configuration model: power-law-ish degree sequence
4 deg_seq = [5,4,3,3,2,2,1,1,1,1]
5 G_cfg = nx.configuration_model(deg_seq, seed=0)
6 G_cfg = nx.Graph(G_cfg) # remove multi-edges
7 G_cfg.remove_edges_from(nx.selfloop_edges(G_cfg))
8 print("cfg degrees:", sorted([d for _,d in G_cfg.degree()], reverse=True))
9
10 # WS small-world sweep
11 for p in [0, 0.05, 0.5, 1.0]:
12     WS = nx.watts_strogatz_graph(30, 4, p, seed=1)
13     print(f"p={p:.2f} C={nx.average_clustering(WS):.3f}"
14           f" L={nx.average_shortest_path_length(WS):.2f}")

```

4.4 Chapter Notes

Erdős and Rényi (1959, 1960) founded random-graph theory; the phase-transition picture via branching processes was clarified by Bollobás (2001). The configuration model was systematised by Bender–Canfield and Molloy–Reed. Watts and Strogatz (1998, *Nature*) sparked the modern small-world revival; Kleinberg (2000) added navigability. Always state whether $G(n, p)$ or $G(n, M)$ is meant—their fluctuations differ but typical properties agree.

4.5 Exercises

- E4.1 **ER degree check.** For $G(200, p = 0.015)$ compute $\langle k \rangle$, the Poisson prediction $P(0)$, and expected number of isolated nodes. Simulate 30 trials and compare.
- E4.2 **Threshold estimation.** Fix $n = 500$, sweep p from 0 to 0.01 (step 0.001), and estimate giant-component fraction S/n (average 20 trials). Plot and identify where S lifts off—compare to $1/n$.
- E4.3 **Configuration model multi-edges.** Generate a configuration model with degrees $[8, 8, 1, 1, 1, 1, 1, 1]$ ($n = 8$). How often do multi-edges or self-loops appear in 100 trials? Why does a hub-heavy sequence increase collision probability?
- E4.4 **WS trade-off.** For $n = 100$, $k = 6$, sweep $p \in \{0, 0.01, 0.05, 0.1, 0.5, 1\}$ and measure $C(p)/C(0)$ and $L(p)/L(0)$. At which p does L drop to half while C stays above $0.5 C(0)$? This is the small-world window.

E4.5 **Null-model test.** Compute modularity Q of the karate club under Louvain. Generate 30 configuration-model randomisations (same degree sequence) and compute their Q values. Is the real Q significant (how many standard deviations above the random mean)?

Chapter 5

Scale-Free Networks & Preferential Attachment

“The rich get richer.” — In citation, Web, and social networks a few hubs hold most links while most nodes have few. This heavy tail is not Poisson noise; it follows a power law born from growth with preference.

From zero: we build here, no power-law background needed. We assume only Ch. 1 (degree distribution) and Ch. 4 (ER, configuration) plus basic probability. Power laws are introduced by picture, then defined, then derived via a generative mechanism.

Learning Outcomes

After this chapter you will be able to:

- (L1) recognise heavy-tailed degree distributions and plot them on log–log axes;
- (L2) define a power law $P(k) \sim k^{-\gamma}$, estimate γ via MLE, and state its cutoff;
- (L3) derive the Barabási–Albert model and show it yields $P(k) \sim k^{-3}$;
- (L4) simulate BA growth, measure hubs, exponent, and age–degree correlation;
- (L5) apply the Clauset–Shalizi–Newman test to critique scale-free claims.

5.1 Heavy Tails and Power Laws

Intuition. In an ER graph with $\langle k \rangle = 6$, seeing a node with $k = 30$ is astronomically unlikely (Poisson tail $\sim e^{-k}$). Yet the Web has pages with millions of in-links while most have a handful. Plotting $P(k)$ on log–log axes turns an exponential into a curve but a power law into a straight line—a visual tell.

Definition 5.1 (Power-law degree distribution). A distribution is (asymptotically) power-law if for $k \geq k_{\min}$,

$$P(k) = C k^{-\gamma}, \quad C = (\gamma - 1) k_{\min}^{\gamma-1}, \quad (5.1)$$

with *exponent* $\gamma > 1$. On log–log, $\log P(k) = \log C - \gamma \log k$ is linear with slope $-\gamma$. Most real “scale-free” networks report $2 < \gamma < 3$, where $\langle k \rangle$ is finite but $\langle k^2 \rangle$ diverges as $k_{\max} \rightarrow \infty$ (variance explodes—hubs dominate).

Why care about γ ? If $2 < \gamma < 3$, the second moment $\langle k^2 \rangle$ grows with system size, which (Ch. 6–7) drives epidemic thresholds to zero and makes random removal harmless but hub targeting devastating.

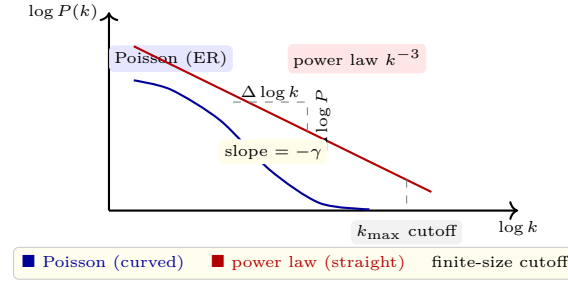


Figure 5.1: Log–log degree distribution. ER (Poisson) curves sharply down (blue); a power law $P(k) \sim k^{-\gamma}$ is a straight line with slope $-\gamma$ (red). Finite networks cut off at $k_{\max}$.

Estimating γ . Do not fit a line by least squares on the log–log histogram—it is biased. The maximum-likelihood estimator for continuous $k \geq k_{\min}$ is $\hat{\gamma} = 1 + n[\sum_i \ln(k_i/k_{\min})]^{-1}$ (Newman, 2005). Choose $k_{\min}$ by minimising the Kolmogorov–Smirnov distance between data and fitted power law. Then test goodness-of-fit (see §5.3).

5.2 Barabási–Albert Model

Intuition. Two ingredients make a power law: *growth* (new nodes keep arriving) and *preferential attachment* (newcomers prefer to link to already-popular nodes). “When a new Web page appears, it is more likely to link to Google than to a personal homepage.” The rich get richer.

Definition 5.2 (Barabási–Albert model). Start with m_0 connected nodes. At each step add one new node with $m \leq m_0$ edges, each attached to an existing node i with probability

$$\Pi(k_i) = \frac{k_i}{\sum_j k_j}. \quad (5.2)$$

After t steps, $n = m_0 + t$, $m = m_0(m_0 - 1)/2 + mt$, and $\langle k \rangle = 2m$.

Theorem 5.1 (BA exponent). In the BA model with m edges per new node, the stationary degree distribution is $P(k) = 2m(m+1)/[k(k+1)(k+2)] \sim 2m^2 k^{-3}$ for $k \geq m$, so $\gamma = 3$ universally (continuum theory: $k_i(t) = m\sqrt{t/t_i}$ where t_i is birth time).

Sketch. Continuum approximation: treat k_i as continuous, $dk_i/dt = m\Pi(k_i) = mk_i/(2mt) = k_i/(2t)$ since $\sum_j k_j = 2mt$. Solve: $dk_i/k_i = dt/(2t)$, so $k_i(t) = m(t/t_i)^{1/2}$. The fraction of nodes with $k_i(t) > k$ equals

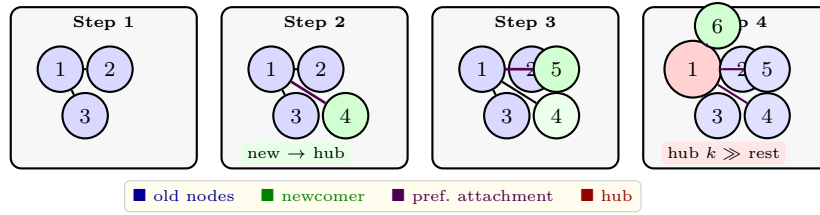


Figure 5.2: BA growth: each new node (green) attaches preferentially to high-degree nodes (violet edges). Early nodes accumulate links and become hubs (red). After many steps, $P(k) \sim k^{-3}$.

$\Pr(t_i < t(m/k)^2) = 1 - (m/k)^2$, differentiate to get $P(k) = 2m^2k^{-3}$. The master-equation proof yields the exact form. □

Consequences: age correlates with degree (old nodes are hubs), $k_{\max} \sim m\sqrt{n}$, and the exponent 3 is robust—but variants change it (fitness, nonlinear attachment, copying).

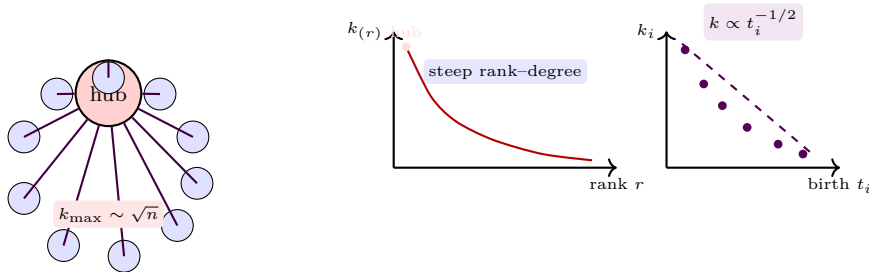


Figure 5.3: Left: BA hub-and-spoke. Middle: rank–degree is steep (few hubs, many leaves). Right: older nodes have larger degree—a testable age–degree correlation.

5.3 Critiquing Scale-Free Claims

Intuition. Many papers claim “our network is scale-free” after eyeballing a straight line on log–log. Clauset, Shalizi, and Newman (2009) showed that rigorous testing rejects most such claims—exponentials, log-normals, and truncated power laws often fit better.

The recipe: (i) estimate $k_{\min}$ and γ by MLE, (ii) compute KS statistic D for goodness-of-fit, (iii) generate synthetic power-law datasets with same n , $k_{\min}$, γ and compare D —a p -value below 0.1 rejects the power-law hypothesis. Even when not rejected, compare against alternatives via likelihood-ratio (Vuong) test. The configuration model with power-law $P(k)$ can then serve as a null for dynamics (Ch. 6–7).

Listing 5.1: BA simulation: growth, exponent, and age–degree correlation.

```

1 import networkx as nx
2 import numpy as np
3
4 G = nx.barabasi_albert_graph(500, 3, seed=0)
5 degrees = np.array([d for _, d in G.degree()])
6 print(f"n=500 m=3 <k>={degrees.mean():.1f} max k={degrees.max()}")
7
8 # Log-log degree histogram (check straight line)
9 import collections
10 cnt = collections.Counter(degrees)
11 ks, pk = zip(*sorted(cnt.items()))

```

```

12 # MLE for gamma (continuous approx, k_min=3)
13 k_min = 3
14 tail = degrees[degrees >= k_min]
15 gamma_hat = 1 + len(tail) / np.sum(np.log(tail / k_min))
16 print(f"MLE gamma ~ {gamma_hat:.2f} (BA predicts 3.0)")
17
18 # Age-degree: node id is birth order in NetworkX BA
19 import matplotlib.pyplot as plt
20 # Hub age check: oldest nodes should have highest degree
21 top5 = sorted(G.degree, key=lambda x: x[1], reverse=True)[:5]
22 print("top hubs (node, k, birth):", [(v, k, v) for v, k in top5])

```

Listing 5.2: PLFit and Clauset test sketch: is this really a power law?

```

1 import networkx as nx
2 import numpy as np
3
4 # Compare Poisson (ER) vs power-law (BA) tail likelihood
5 ER = nx.erdos_renyi_graph(500, 0.012, seed=1)
6 BA = nx.barabasi_albert_graph(500, 3, seed=1)
7 for name, G in [("ER", ER), ("BA", BA)]:
8     degs = np.array([d for _, d in G.degree()])
9     # Simple tail check: fraction with k > 2<k>
10    frac = (degs > 2*degs.mean()).mean()
11    print(f"{name}: <k>={degs.mean():.1f} P(k>2<k>)= {frac:.3f}"
12          f" max={degs.max()}")
13 # ER tail fraction ~ Poisson (tiny); BA tail heavy (large)
14 # For rigorous test use 'powerlaw' package: powerlaw.Fit(degs)

```

5.4 Chapter Notes

Price (1965) first modelled cumulative advantage for citations; Barabási and Albert (1999, *Science*) popularised preferential attachment for networks. The exact $P(k) \sim k^{-3}$ solution is due to Krapivsky et al. and Dorogovtsev–Mendes. Variants (Bianconi–Barabási fitness, copying, redirection, nonlinear $\Pi \sim k^\alpha$) tune γ or break scale-free entirely. Clauset et al. (2009) and Broido & Clauset (2019) provide the modern statistical critique—“scale-free networks are rare” when tested rigorously.

5.5 Exercises

- E5.1 **Log–log by hand.** Given degrees $[1, 1, 1, 2, 2, 5, 9, 14]$, plot $P(k)$ vs k on log–log (sketch). Does a Poisson or power law seem more plausible? Estimate γ crudely from the slope between $k = 2$ and $k = 14$.
- E5.2 **Continuum derivation.** Starting from $dk_i/dt = k_i/(2t)$, $k_i(t_i) = m$, integrate to get $k_i(t) = m\sqrt{t/t_i}$. Then show $\Pr(k_i(t) > k) = (m/k)^2$ and differentiate to obtain $P(k) \sim k^{-3}$.
- E5.3 **Simulate and measure.** Generate $\text{BA}(n = 1000, m = 2)$ and compute degree histogram. Fit γ via MLE with $k_{\min} = m$ and report. Does doubling m change $\hat{\gamma}$? Explain.

- E5.4 **Age–degree.** In your BA graph, compute Pearson correlation between node birth time (id) and final degree. Is it negative as predicted? Compare with an ER graph of same n , $\langle k \rangle$.
- E5.5 **Clauset test.** Use the ‘powerlaw’ package (or hand KS) to test whether a BA(500, 3) degree sequence is plausibly power-law ($p > 0.1$) and whether an ER(500, 0.012) sequence is rejected. Report $k_{\min}$, $\hat{\gamma}$, and p -values.

Chapter 6

Dynamics & Spreading: SIR on Networks

“Ideas, like viruses, ride the links.” — Whether a rumour dies out or sweeps the world depends not only on how infectious it is but on the wiring it rides.

From zero: we build here, no epidemiology background needed. We assume only Ch. 1–2 (graph, degree, spectra) and basic probability. SIR, SIS, and threshold models are introduced by story, then formalised, then simulated.

Learning Outcomes

After this chapter you will be able to:

- (L1) define SI, SIS, and SIR models on networks (states, rates, transitions);
- (L2) derive the epidemic threshold λ_c via mean-field and via spectral radius;
- (L3) simulate SIR on ER, small-world, and scale-free graphs and compare outbreaks;
- (L4) evaluate immunisation strategies: random, degree-targeted, and betweenness-targeted;
- (L5) describe threshold/cascade models and opinion dynamics as extensions.

6.1 Compartment Models on Graphs

Intuition. Each person (node) is in a state: *Susceptible* (healthy, can catch), *Infected* (has it, can spread), *Recovered* (immune, out). In continuous time, an infected node infects each susceptible neighbour at rate β and recovers at rate μ . The ratio $\lambda = \beta/\mu$ is the *effective spreading rate*. If λ is tiny, the outbreak fizzles; if large, it explodes—where is the boundary?

Definition 6.1 (SIR, SIS, SI on networks). States per node: $S \rightarrow I \rightarrow R$ (SIR, permanent immunity), $S \rightarrow I \rightarrow S$ (SIS, can be reinfected), $S \rightarrow I$ (SI, never recovers). For SIR, let $s_v(t)$, $i_v(t)$, $r_v(t)$ be probabilities

node v is in each state ($s + i + r = 1$). Mean-field (quenched) dynamics:

$$\dot{i}_v = \beta s_v \sum_u A_{vu} i_u - \mu i_v, \quad \dot{s}_v = -\beta s_v \sum_u A_{vu} i_u, \quad \dot{r}_v = \mu i_v. \quad (6.1)$$

SIS drops the R equation and adds $+\mu i_v$ to $\dot{s}_v$. SI sets $\mu = 0$.

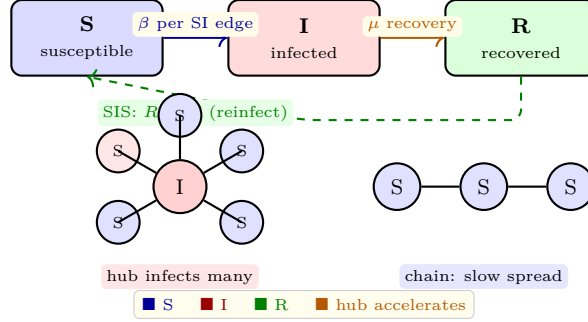


Figure 6.1: Compartment models and why hubs matter. Top: $S \rightarrow I$ at rate β per SI edge, $I \rightarrow R$ at μ ; dashed is SIS reinfection. Bottom: a star hub reaches many susceptibles at once (fast), a chain spreads one hop at a time (slow).

6.2 Epidemic Threshold

Intuition. If each infected node on average infects less than one new node before recovering, the outbreak shrinks; if more than one, it grows. On a network, “average” depends on who you are connected to—hubs amplify, leaves dampen. The tipping point is an eigenvalue.

Theorem 6.1 (Spectral threshold for SIS/SIR). For the quenched mean-field model with adjacency A and spectral radius $\rho(A) = \lambda_{\max}(A)$,

$$\lambda_c = \frac{\beta}{\mu} \Big|_{\text{crit}} = \frac{1}{\rho(A)}. \quad (6.2)$$

If $\lambda < \lambda_c$, the disease-free state is stable (outbreak dies); if $\lambda > \lambda_c$, endemic/explosive spread occurs. For annealed (degree-based) mean-field, $\lambda_c^{\text{HMF}} = \langle k \rangle / \langle k^2 \rangle$.

Idea. Linearise around disease-free ($s_v \approx 1$, $i_v \approx 0$): $\dot{i} = (\beta A - \mu I)i$. Growth is governed by the largest eigenvalue of $\beta A - \mu I$, namely $\beta \rho(A) - \mu$. It turns positive exactly when $\beta/\mu > 1/\rho(A)$. Heterogeneous mean-field averages over degree classes to get $\langle k^2 \rangle / \langle k \rangle$. $\square$ $\square$

Consequences tie directly to Ch. 5: for scale-free $P(k) \sim k^{-\gamma}$ with $2 < \gamma < 3$, $\langle k^2 \rangle$ diverges, so $\lambda_c^{\text{HMF}} \rightarrow 0$ as $n \rightarrow \infty$ —arbitrarily weak viruses persist (no threshold in the infinite limit). ER graphs have finite $\langle k^2 \rangle$, so a genuine threshold remains.

6.3 Immunisation and Influence

Intuition. Vaccinating a random person protects one node; vaccinating a hub protects many paths. To stop an outbreak cheaply, target high-degree or high-betweenness nodes (acquaintance immunisation picks a

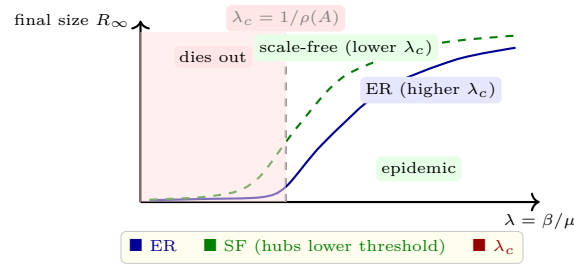


Figure 6.2: Epidemic threshold. Below λ_c final recovered fraction is near zero; above it jumps. Scale-free networks (green, hubs) have smaller λ_c than ER (blue) of same $\langle k \rangle$ because $\rho(A)$ is larger when hubs exist.

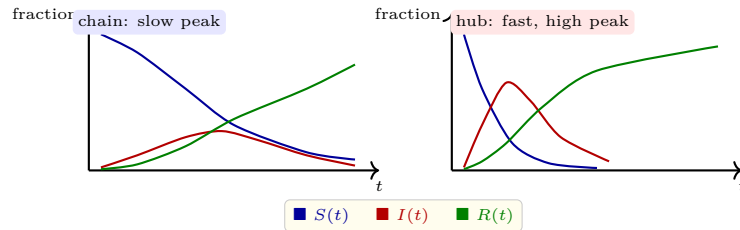


Figure 6.3: SIR curves $S(t), I(t), R(t)$ on a chain (left: broad, low peak) vs a star-like network (right: sharp, high peak). Hubs synchronise infections—the epidemic is faster and larger.

random neighbour—biased toward hubs—when global ranking is unknown). The same logic flipped is *influence maximisation*: seed the best spreaders to maximise adoption.

Listing 6.1: SIR simulation on ER vs BA: threshold and final size.

```

1 import networkx as nx
2 import numpy as np
3
4 def sir_final_size(G, beta=0.3, mu=0.2, seed=0):
5     # Simple discrete-time SIR (parallel update)
6     import random; random.seed(seed)
7     infected = {next(iter(G.nodes()))}
8     recovered = set(); susceptible = set(G.nodes()) - infected
9     while infected:
10         new_inf = set()
11         for v in list(infected):
12             for nb in G.neighbors(v):
13                 if nb in susceptible and random.random() < beta:
14                     new_inf.add(nb)
15             if random.random() < mu:
16                 recovered.add(v); infected.remove(v)
17         susceptible -= new_inf; infected |= new_inf
18         if len(recovered) > 5 * G.number_of_nodes(): break
19     return len(recovered) / G.number_of_nodes()
20
21 for name, G in [("ER", nx.erdos_renyi_graph(200, 0.03, seed=1)),
22                ("BA", nx.barabasi_albert_graph(200, 3, seed=1))]:
23     rho = max(np.linalg.eigvals(nx.to_numpy_array(G)).real)
24     print(name, f"rho={rho:.1f} lambda_c~{1/rho:.3f}"
25           f" final R={sir_final_size(G, beta=0.25, mu=0.2):.2f}")

```

Listing 6.2: Targeted vs random immunisation: removing hubs collapses spreading.

```

1 import networkx as nx
2
3 def giant_after_removal(G, remove):
4     H = G.copy(); H.remove_nodes_from(remove)
5     if not H: return 0
6     return max(len(c) for c in nx.connected_components(H)) / G.number_of_nodes()
7
8 G = nx.barabasi_albert_graph(200, 3, seed=0)
9 # Random vs degree-targeted removal of 10 nodes
10 import random; random.seed(0)
11 rand10 = random.sample(list(G.nodes()), 10)
12 hub10 = sorted(G.degree, key=lambda x: x[1], reverse=True)[:10]
13 hub10 = [v for v, _ in hub10]
14 print(f"random 10 removal: giant={giant_after_removal(G, rand10):.2f}")
15 print(f"hub 10 removal: giant={giant_after_removal(G, hub10):.2f}")
16 # Hub removal shatters scale-free networks (Ch.7 preview)

```

6.4 Beyond SIR: Thresholds, Cascades, Opinions

Not all spreading is simple contagion (one contact can infect). In *complex contagion* a node adopts only when a fraction ϕ of neighbours have adopted—capturing social reinforcement. The *linear threshold* and *independent cascade* models formalise this; influence maximisation becomes NP-hard but greedy gives a $(1 - 1/e)$ approximation (submodularity). Opinion dynamics (voter, DeGroot) add continuous states. All run on the same wiring—topology still governs speed and reach.

6.5 Chapter Notes

Kermack and McKendrick (1927) founded compartmental epidemics; Pastor-Satorras and Vespignani (2001) linked the threshold to $\langle k^2 \rangle / \langle k \rangle$ on scale-free nets; Chakrabarti et al. (2008) gave the spectral form $\lambda_c = 1/\rho(A)$. Discrete-time SIR used in code follows the Gillespie / synchronous update family. Immunisation surveys: Pastor-Satorras et al. (2015). For influence maximisation, Kempe–Kleinberg–Tardos (2003) is the landmark.

6.6 Exercises

- E6.1 **SI on a star.** Consider SI ($\mu = 0$) on a star with centre 1 and leaves $2, \dots, n$. Start with centre infected. Derive expected time to infect all leaves if each SI edge transmits with probability β per step (discrete time, independent).
- E6.2 **Threshold comparison.** For ER with $n = 500$, $p = 0.01$ and BA with $n = 500$, $m = 3$ (same $\langle k \rangle \approx 6$), estimate $\rho(A)$ numerically and predict λ_c . Simulate SIR at $\lambda = 0.8\lambda_c$ and $1.2\lambda_c$ and compare final sizes.
- E6.3 **Mean-field vs spectral.** Compute $\langle k \rangle$, $\langle k^2 \rangle$, and $\rho(A)$ for a 100-node BA graph. Compare $\lambda_c^{\text{HMF}} = \langle k \rangle / \langle k^2 \rangle$ vs $1/\rho(A)$. Which is smaller? Why does HMF underestimate the threshold on clustered graphs?

- E6.4 **Immunisation sweep.** On BA(200,3), remove nodes in random order vs degree-descending order (5 at a time, up to 40). Plot giant-component fraction vs number removed. At what removal fraction does targeted attack halve the giant?
- E6.5 **Complex contagion.** Simulate linear-threshold with $\phi = 0.4$ on a 30-node WS graph ($k = 4$, $p = 0.1$). Seed two adjacent vs two distant nodes and report final adoption. Why does clustering help complex but hinder simple contagion?

Chapter 7

Network Robustness & Resilience

“How much can you break before it falls apart?” — Random failures chip at networks daily; targeted attacks aim at hubs. Some networks shrug off the first and shatter under the second.

From zero: we build here, no resilience background needed. We assume only Ch. 1 (components, degree) and Ch. 4–5 (ER, scale-free). Robustness is introduced by picture, then measured, then designed for.

Learning Outcomes

After this chapter you will be able to:

- (L1) define robustness via giant-component size $S(f)$ under removal fraction f ;
- (L2) contrast random failures vs targeted attacks on ER and scale-free graphs;
- (L3) derive percolation thresholds p_c and the critical fraction f_c ;
- (L4) explain why scale-free nets are robust-yet-fragile and design onion-hardened variants;
- (L5) model cascading failures and interdependent-network collapse.

7.1 Measuring Robustness: $S(f)$

Intuition. Remove a fraction f of nodes (or edges) and watch the largest connected component $S(f)$. If S shrinks gracefully, the network is robust; if it cliffs to near zero at some f_c , it is fragile. Two removal orders matter: *random* (failures: each node equally likely) and *targeted* (attacks: highest-degree first).

Definition 7.1 (Robustness curve and f_c). Let $S(f) = |C_{\max}(f)|/n$ after removing fraction f of nodes. The *percolation (critical) threshold* f_c is the smallest f where $S(f) \rightarrow 0$ (no giant remains). For random removal with occupation probability $p = 1 - f$, $f_c = 1 - p_c$ where p_c is the percolation point. Area under $S(f)$ (AUC) is a summary robustness score in $[0, 1]$.

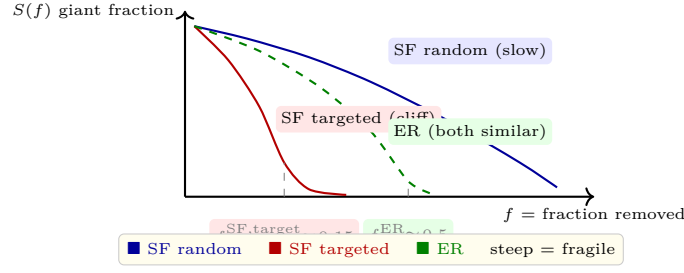


Figure 7.1: Robustness curves $S(f)$. Scale-free (SF) under random removal decays slowly—robust. Under hub-targeted attack it cliffs early ($f_c \approx 0.15$)—fragile. ER decays similarly for both (no hubs to target). AUC summarises overall resilience.

7.2 Percolation Thresholds

Intuition. Percolation asks: if each node is occupied with probability p (kept with p , removed with $1 - p$), when does a spanning (giant) cluster appear? The answer depends on moments of $P(k)$.

Theorem 7.1 (Molloy–Reed criterion and p_c). A random graph with degree distribution $P(k)$ has a giant component iff $\langle k^2 \rangle / \langle k \rangle > 2$. Under random occupation with probability p , the threshold is

$$p_c = \frac{\langle k \rangle}{\langle k^2 \rangle - \langle k \rangle}, \quad f_c = 1 - p_c = 1 - \frac{\langle k \rangle}{\langle k^2 \rangle - \langle k \rangle}. \quad (7.1)$$

For ER with $\langle k \rangle$, $\langle k^2 \rangle = \langle k \rangle^2 + \langle k \rangle$, so $p_c = 1/\langle k \rangle$ and $f_c = 1 - 1/\langle k \rangle$ (e.g. $\langle k \rangle = 4 \Rightarrow f_c = 0.75$ for random removal, but only for ER). For scale-free $2 < \gamma < 3$, $\langle k^2 \rangle$ diverges, so $p_c \rightarrow 0$: the giant survives almost any random removal.

Idea. Branching-process as in Ch. 4 but with occupation: following a random edge, the expected number of new occupied neighbours is $p(\langle k^2 \rangle / \langle k \rangle - 1)$. Giant exists when this exceeds 1, giving p_c . $\square$ $\square$

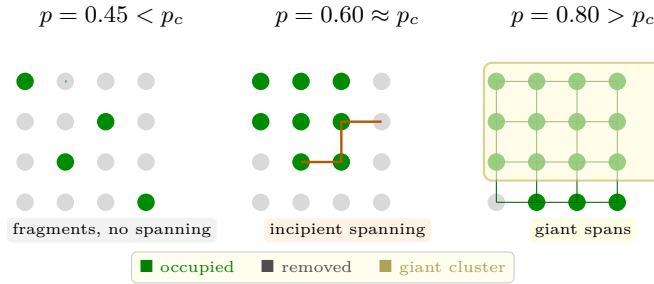


Figure 7.2: Site percolation on a 2-D lattice. Below p_c only small clusters (left); near p_c a fragile spanning cluster appears (middle); above p_c the giant dominates (right, yellow). Network percolation is the irregular-graph analogue.

7.3 Robust Yet Fragile and Hardening

Scale-free networks are *robust yet fragile* (Doyle): random removal likely hits leaves ($f_c \approx 1$), but removing a few hubs collapses the giant. ER, with no hubs, treats random and targeted similarly.

Hardening strategies: (i) *onion structure*: sort nodes by degree, link high-degree nodes in layers so the core survives hub removal; (ii) *redundancy / bypass*: add edges that shortcut hubs; (iii) *modular hardening*: reinforce inter-module bridges (Ch. 3) rather than just hubs, since bridges control $S(f)$ when communities are strong.

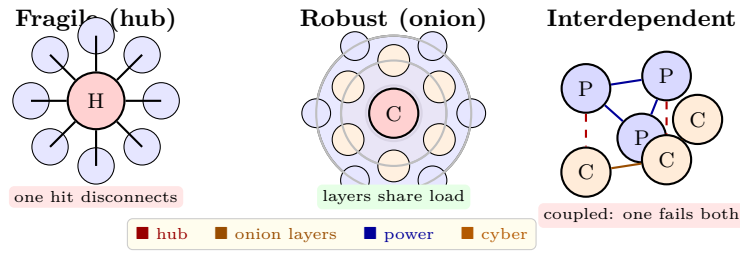


Figure 7.3: Left: hub-centric is fragile. Middle: onion (concentric degree layers) is hardened—core survives. Right: interdependent networks (power P + cyber C , dashed dependencies) cascade: one failure propagates back and forth.

Cascades and interdependence. In a power grid, a line failure redistributes load and can overload neighbours (cascade). In coupled networks (power + communication, Fig. 7.3 right), a power node needs a cyber node to operate and vice versa. Buldyrev et al. (2010) showed the percolation becomes first-order (abrupt, discontinuous)—far more catastrophic than single-network second-order decay.

Listing 7.1: Robustness curves: random vs hub-targeted removal on BA vs ER.

```

1 import networkx as nx
2 import random
3
4 def robustness_curve(G, order):
5     H = G.copy(); n = G.number_of_nodes()
6     giants = []
7     for v in order:
8         if v in H: H.remove_node(v)
9         if not H: giants.append(0); break
10        giant = max(len(c) for c in nx.connected_components(H))
11        giants.append(giant / n)
12    return giants
13
14 G_sf = nx.barabasi_albert_graph(200, 3, seed=0)
15 G_er = nx.erdos_renyi_graph(200, 0.03, seed=0)
16 random.seed(0)
17 for name, G in [("BA", G_sf), ("ER", G_er)]:
18     rand_order = random.sample(list(G.nodes()), 50)
19     hub_order = sorted(G.degree, key=lambda x: x[1], reverse=True)
20     hub_order = [v for v, _ in hub_order[:50]]
21     r = robustness_curve(G, rand_order)
22     h = robustness_curve(G, hub_order)
23     print(f"{name} S after 20 removals: random {r[19]:.2f} targeted {h[19]:.2f}")

```

Listing 7.2: Percolation threshold check via moment formula.

```

1 import networkx as nx
2 import numpy as np
3
4 def percolation_stats(G):

```

```

5     degs = np.array([d for _, d in G.degree()])
6     k1, k2 = degs.mean(), (degs**2).mean()
7     pc = k1 / (k2 - k1) if k2 > k1 else 1.0
8     return k1, k2, pc, 1-pc
9
10    for name, G in [("ER p=0.03", nx.erdos_renyi_graph(200,0.03,seed=1)),
11                   ("BA m=3", nx.barabasi_albert_graph(200,3,seed=1))]:
12        k1,k2,pc,fc = percolation_stats(G)
13        print(f"{name}: <k>={k1:.1f} <k2>={k2:.1f} pc={pc:.3f} fc={fc:.3f}"
14              f" robust-random? {fc>0.8}")

```

7.4 Chapter Notes

Albert, Jeong, and Barabási (2000, *Nature*) first demonstrated robust-yet-fragile on the Web and Internet. Cohen et al. (2000, 2001) derived p_c for scale-free nets. Schneider et al. (2011) introduced onion hardening. Buldyrev et al. (2010) discovered first-order percolation in interdependent networks, reshaping infrastructure-risk thinking. For design, Valente–Latora–Marchiori efficiency and Schneider’s R index complement $S(f)$.

7.5 Exercises

- E7.1 **Handshake percolation.** For ER with $n = 1000$, $\langle k \rangle = 4$, compute p_c and f_c for random site percolation. Simulate random removal of $f = 0.2, 0.5, 0.7$ fractions (20 trials each) and report mean $S(f)$. Where does the giant vanish?
- E7.2 **Hub targeting.** On BA($n = 300$, $m = 2$) remove the top 10 hubs vs 10 random nodes (10 trials for random). Report mean S after removal. Repeat for ER($n = 300$, $p = 0.013$) with same $\langle k \rangle$. Which contrast is larger and why?
- E7.3 **Moment formula.** Given degree sequence $[1, 1, 1, 2, 2, 2, 6, 6]$, compute $\langle k \rangle$, $\langle k^2 \rangle$, and p_c via the moment formula. Does a giant exist at $p = 1$? At $p = 0.5$?
- E7.4 **Onion gain.** Starting from BA(200,3), add 20 “onion” edges linking high-degree nodes in layers (pair the top 10 hubs in a cycle, then next 10). Re-measure $S(f)$ under targeted attack ($f = 0.1$). Does the hardened network retain a larger giant? By how much?
- E7.5 **Cascade sketch.** Two coupled ER graphs (100 nodes each, $p = 0.04$) with one-to-one dependency (power i depends on cyber i). Simulate: remove 10 random power nodes, then alternately prune nodes that lack a giant connection or a live dependency. Report final surviving fraction. Compare to single-network removal of 10 nodes.

Chapter 8

Applications: Social, Biological & Technological Networks

“The same math in different clothes.” — Friendship, protein binding, and router links obey the same network laws yet reveal domain-specific stories. Seeing one lens through three domains cements the whole picture.

From zero: we build here, no domain background needed. We assume only Ch. 1–7 (graph, centrality, community, random/scale-free, spreading, robustness). Each domain is introduced from scratch, then analysed with tools from earlier chapters.

Learning Outcomes

After this chapter you will be able to:

- (L1) map and interpret social networks: homophily, triadic closure, and influence;
- (L2) analyse biological networks: PPI hubs as essential proteins, metabolic and neural motifs;
- (L3) dissect technological networks: WWW bow-tie, Internet AS-graph, power-grid fragility;
- (L4) synthesise universal vs domain-specific patterns across the three domains;
- (L5) conduct a capstone mini-project: collect, measure, visualise, and report on a real dataset.

8.1 Social Networks

Intuition. People befriend those nearby and those like themselves (*homophily*), and friends of friends tend to become friends (*triadic closure*—closing a triangle). These micro-mechanisms create high clustering and communities by age, interest, or workplace. A collaboration network (authors linked if they co-authored) is bipartite authors–papers projected onto authors; its communities are research fields.

Standard traits: heavy-tailed degrees (a few super-connectors), high C , positive degree assortativity (hubs connect to hubs), and community structure. Centrality here predicts influence; spreading models capture rumour or adoption (Ch. 6 threshold models fit better than simple SIR because social adoption often needs reinforcement).

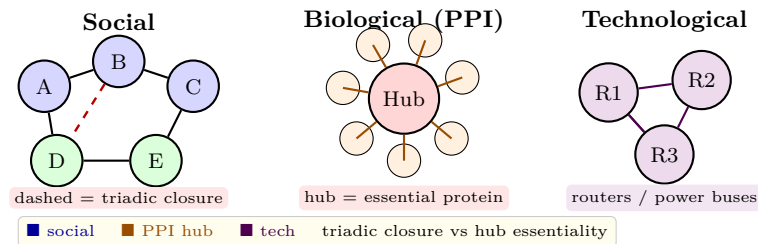


Figure 8.1: Three domains, same language. Social: communities and triadic closure (red dashed is a triangle closing). Biological: a few hub proteins bind many partners—hubs are essential. Technological: router/power grids are engineered, more regular but still with hubs.

8.2 Biological Networks

Intuition. In a protein–protein interaction (PPI) network, nodes are proteins, edges are bindings. Removing a hub protein often kills the cell (lethality–centrality rule), while removing a leaf does not—a direct analogue of Ch. 7 targeted attack. Metabolic networks are bipartite (metabolites + reactions) and hierarchical; neural networks (e.g. *C. elegans* 302 neurons) are small-world with modules for function and a rich club of hub neurons interconnecting modules.

Traits that differ from social: PPI is disassortative (hubs avoid hubs, to separate functions), clustering is moderate, and motifs matter—feed-forward loops and bifans appear far more than in random nulls (use configuration model with same degrees as null, as in Ch. 4).

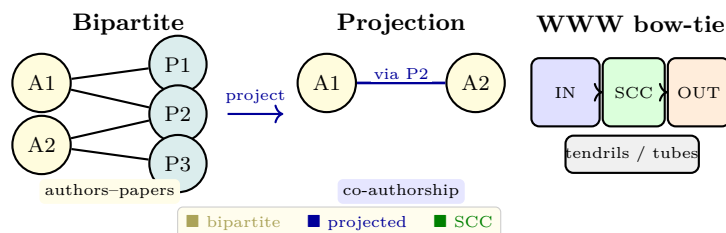


Figure 8.2: Left: bipartite author–paper network projected onto co-authorship (shared paper = edge). Right: the WWW bow-tie (Broder et al.)—IN can reach SCC, SCC can reach OUT, tendrils hang off. Each domain reuses the same Ch. 1–7 machinery.

8.3 Technological Networks

Intuition. The Web graph is directed (hyperlinks), heavy-tailed in-degree, and bow-tie shaped (Fig. 8.2 right): a strongly connected core (SCC) where every page reaches every other, an IN that can reach the core, an OUT reachable from the core, plus tendrils. PageRank (Ch. 2) was invented for this graph. The Internet’s autonomous-system (AS) graph is scale-free and disassortative; power grids are more lattice-like, with exponential degree tails—engineered for redundancy, yet vulnerable to cascades (Ch. 7).

8.4 Universal vs Specific and Capstone

Universals across domains: heavy-tailed degrees (often, but test via Ch. 5 Clauset), small-world (high C , short L), communities (Ch. 3), and hub-driven spreading/robustness (Ch. 6–7). Specifics: social is assortative and clustered; bio is disassortative and motif-rich; tech is engineered—Web is directed and bow-tie, power grid is not scale-free. Recognising both prevents over-generalising “everything is scale-free.”

Capstone workflow. Pick a dataset (Zachary karate, Les Misérables co-appearance, *C. elegans* neural, or SNAP power grid). Then: collect/clean $\rightarrow$ measure ($n, m, \langle k \rangle, P(k), C, L$) $\rightarrow$ null test (ER / configuration) $\rightarrow$ centrality + community $\rightarrow$ simulate SIR and robustness $\rightarrow$ visualise $\rightarrow$ report. The exercises scaffold this pipeline.

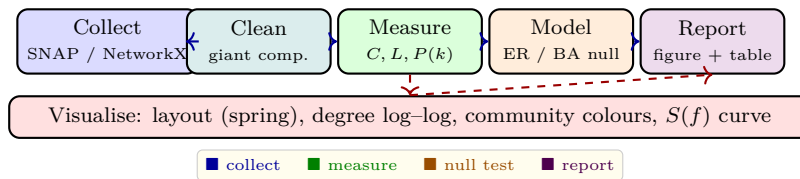


Figure 8.3: Capstone pipeline: from raw data to report. Each stage maps to a chapter: measure (Ch. 1–2), null model (Ch. 4–5), spreading/robustness (Ch. 6–7), applications (this chapter). Visualisation threads through all.

Listing 8.1: End-to-end mini-project on Les Misérables co-appearance network.

```

1 import networkx as nx
2
3 G = nx.les_miserables_graph()
4 print(f"LesMis: n={G.number_of_nodes()} m={G.number_of_edges()}")
5     f" <k>={2*G.number_of_edges()/G.number_of_nodes():.1f}")
6 print(f"C={nx.average_clustering(G):.3f}")
7     f" L={nx.average_shortest_path_length(G):.2f}")
8 # Degree heavy tail?
9 import numpy as np
10 degs = np.array([d for _, d in G.degree()])
11 print("max k:", degs.max(), "P(k>10):", (degs>10).mean())
12 # Communities (Louvain) and central figure
13 import community as community_louvain
14 part = community_louvain.best_partition(G)
15 print("communities:", len(set(part.values())),
16     "Q=", round(community_louvain.modularity(part, G), 3))
17 bet = nx.betweenness_centrality(G)
18 hub = max(bet, key=bet.get)
19 print("top betweenness:", hub, G.nodes[hub], round(bet[hub], 3))

```

Listing 8.2: Cross-domain comparison: karate (social) vs power grid (tech).

```

1 import networkx as nx
2
3 # Karate (social: assortative, high C) vs synthetic grid-like
4 karate = nx.karate_club_graph()
5 # Approximate grid trait: WS lattice-like
6 grid_like = nx.watts_strogatz_graph(100, 4, 0.02, seed=1)

```



```

7
8 for name, G in [("karate", karate), ("WS-grid", grid_like)]:
9     print(f"\n{name}: n={G.number_of_nodes()} m={G.number_of_edges()}")
10         f" C={nx.average_clustering(G):.3f}"
11         f" assort={nx.degree_assortativity_coefficient(G):.2f}")
12     # Robustness hint: spectral radius (threshold proxy)
13     import numpy as np
14     rho = max(np.linalg.eigvals(nx.to_numpy_array(G)).real)
15     print(f" rho={rho:.1f} -> lambda_c~{1/rho:.3f}")

```

Remark 8.1 (Ethics and privacy). Network data are sensitive: de-anonymisation via structure alone can re-identify individuals (Narayanan–Shmatikov). Biological and infrastructure data have dual-use risks. Always anonymise, aggregate, and obtain consent; respect SNAP/Twitter/protein-data licences and report limitations. For infrastructure, coordinate disclosure before publishing vulnerabilities.

Example 8.1 (Karate club as a microcosm). Zachary’s karate club (34 nodes, 78 edges) will be your running testbed. Its two ground-truth factions (“Mr. Hi” vs “John A”) let you evaluate community detection (Ch. 3 NMI/ARI), its hubs let you test centrality (Ch. 2: instructor has degree 17), and its $C = 0.588$, $L = 2.41$ illustrate small-world (Ch. 4: vs ER $C \approx 0.14$, $L \approx 2.2$ with same n, m). Every capstone dataset can be compared to this baseline for intuition before formal tests. Use it to debug your pipeline before scaling to thousands of nodes.

Remark 8.2 (Assortativity matters). Social networks are typically assortative (hubs link to hubs, $r > 0$); biological and technological nets are often disassortative ($r < 0$) because hubs avoid each other to compartmentalise function or cost. Assortativity shapes robustness: assortative nets retain a core under random removal but fragment differently under targeting.

8.5 Chapter Notes

Social-network analysis was founded by Moreno (1934) and Milgram (1967) “six degrees”; modern treatments: Easley–Kleinberg (2010). PPI lethality–centrality: Jeong et al. (2001); *C. elegans* connectome: Watts–Strogatz (1998) and Varshney et al. (2011). WWW bow-tie: Broder et al. (2000); Internet AS-graph: Faloutsos et al. (1999); power-grid small-world: Watts (1999). For motifs, see Milo et al. (2002). SNAP (Leskovec) and NetworkX provide the datasets used above; always cite the source.

8.6 Exercises

E8.1 LesMis full report. Run the LesMis snippet, then additionally compute degree distribution $P(k)$, plot log-log, estimate γ via MLE, and compare C and L to an ER graph with same n, m . Is it small-world? Write a 1-page summary with one figure and one table.

E8.2 PPI hub meaning. Load a PPI network (e.g. BioGRID yeast via NetworkX ‘powerlaw’ example or SNAP). Find the top-5 hubs by degree and by betweenness. Are they the same? Look up one hub protein—is it known to be essential? Discuss lethality–centrality.

- E8.3 **Bow-tie check.** Build a directed 8-node Web toy with a 3-node SCC, 2 IN nodes, 2 OUT nodes, and 1 tendril. Identify each node's bow-tie region via reachability (BFS forward+reverse). What fraction of pairs are mutually reachable?
- E8.4 **Bipartite projection.** From the karate bipartite view (members $\times$ events), or authors $\times$ papers (Fig. 8.2), project onto the second mode and compare clustering before vs after projection. Why does projection inflate C ?
- E8.5 **Capstone dataset paper.** Choose one SNAP dataset not used above (e.g. email-EuAll, powerGrid, celegans). Carry the full pipeline of Fig. 8.3: measure, null test, centrality, community, SIR and robustness sweep. Deliver a 2-page report with 3 figures and a table; state one universal and one domain-specific finding.